

Домашнее задание № 7

1. Создание таблицы

```
CREATE TABLE user_actions (  
    user_id UInt64,  
    action String,  
    expense UInt64  
) ENGINE = MergeTree()  
ORDER BY (user_id, action);
```

2. Создание файла users.csv для словаря

user_id,email

1,user1@example.com

2,user2@example.com

3,user3@example.com

4,user4@example.com

5,user5@example.com

3. Создание словаря из csv-файла

```
CREATE DICTIONARY users_dict (  
    user_id UInt64,  
    email String  
)  
PRIMARY KEY user_id  
SOURCE(FILE(  
    path '/var/lib/clickhouse/user_files/users.csv'  
    format 'CSVWithNames'  
))  
LIFETIME(MIN 300 MAX 360)  
LAYOUT(HASHED());
```

4. Проверка словаря

The screenshot shows a SQL query execution interface. At the top, there is a comment "-- Проверка словаря" (Dictionary check). Below it is the query: `SELECT dictGet('users_dict', 'email', toUInt64(1));`. The interface shows the query was executed successfully. Below the query, there is a section labeled "результат 1" (result 1) which displays the output of the query. The output is a single row with the value "user1@example.com".

```
-- Проверка словаря  
SELECT dictGet('users_dict', 'email', toUInt64(1));
```

результат 1 ×

`SELECT dictGet('users_dict', 'email', toUInt64(1))` | Введите SQL выражение чт

	A-Z dictGet('users_dict', 'email', toUInt64(1))
1	user1@example.com

5. Наполнение таблицы данными

INSERT INTO user_actions VALUES

```
(1, 'login', 0),
(1, 'click', 10),
(1, 'purchase', 100),
(1, 'click', 15),
(1, 'logout', 0),
(2, 'login', 0),
(2, 'click', 20),
(2, 'click', 30),
(2, 'purchase', 150),
(2, 'logout', 0),
(3, 'login', 0),
(3, 'click', 5),
(3, 'purchase', 200),
(3, 'click', 25),
(3, 'logout', 0),
(4, 'login', 0),
(4, 'click', 40),
(4, 'purchase', 300),
(4, 'click', 35),
(4, 'logout', 0),
(5, 'login', 0),
(5, 'click', 50),
(5, 'click', 60),
(5, 'purchase', 400),
(5, 'logout', 0);
```

6. Запрос с dictGet, аккумулятивной суммой и сортировкой

```
SELECT
  dictGet('users_dict', 'email', user_id) AS email,
  user_id,
  action,
  expense,
  sum(expense) OVER (PARTITION BY action ORDER BY user_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_expense
FROM user_actions
ORDER BY email, user_id, action;
```

user_actions 1					
SELECT dictGet('users_dict', 'email', user_id) AS email, user_id, action, expense, sum(expense) OVER (PARTITION BY action ORDER BY user_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_expense FROM user_actions ORDER BY email, user_id, action;					
	A-Z email	123 user_id	A-Z action	123 expense	123 cumulative_expense
1	user1@example.com	1	click	10	10
2	user1@example.com	1	click	15	25
3	user1@example.com	1	login	0	0
4	user1@example.com	1	logout	0	0
5	user1@example.com	1	purchase	100	100
6	user2@example.com	2	click	20	45
7	user2@example.com	2	click	30	75
8	user2@example.com	2	login	0	0
9	user2@example.com	2	logout	0	0
10	user2@example.com	2	purchase	150	250
11	user3@example.com	3	click	5	80
12	user3@example.com	3	click	25	105
13	user3@example.com	3	login	0	0
14	user3@example.com	3	logout	0	0